

CAA Lab Assignment 5

```
int prediction_mode = ONE_BIT_BP;

struct CPU {
    unsigned char R[4];
    unsigned char ACC;
    unsigned char PC;
    unsigned char IR;
    unsigned char Z, C;
};

struct IFEX {
    unsigned char IR;
    unsigned char PC;
    int valid;
    int predicted_taken;
};

struct BPEntry {
    int valid;
    int prediction;
};

unsigned char IMEM[MEM_SIZE];
unsigned char DMEM[MEM_SIZE];
CPU cpu;
IFEX ifex;
BPEntry bp_table[BP_SIZE];

int cycles = 0;
int instructions = 0;
int total_branches = 0;
int correct_predictions = 0;
int mispredictions = 0;
int branch_penalty_cycles = 0;

void reset_cpu() {
    memset(&cpu, 0, sizeof(cpu));
    memset(&ifex, 0, sizeof(ifex));
}
```

```

    memset(bp_table, 0, sizeof(bp_table));
    cpu.IE = 1;
    cpu.R[1] = 1;
    cpu.R[2] = 0;
}

void load_program(const string& fname) {
    ifstream fin(fname);
    if (!fin) {
        cout << "Cannot open program file\n";
        exit(1);
    }
    int addr = 0;
    unsigned int x;
    while (fin >> hex >> x)
        IMEM[addr++] = (unsigned char)x;
    fin.close();
}

int is_branch(unsigned char ir) {
    unsigned char op = (ir >> 4) & 0xF;
    return (op == 0xA || op == 0xB);
}

int branch_taken(unsigned char ir) {
    unsigned char op = (ir >> 4) & 0xF;
    return (op == 0xA) || (op == 0xB && cpu.Z);
}

int bp_index(unsigned char pc) {
    return pc % BP_SIZE;
}

int predict_branch(unsigned char pc) {
    if (prediction_mode == ONE_BIT_BP) {
        int idx = bp_index(pc);
        if (bp_table[idx].valid)
            return bp_table[idx].prediction;
    }
    return 0;
}

void update_predictor(unsigned char pc, int taken) {

```

```

if (prediction_mode  $\neq$  ONE_BIT_BP)
    return;

int idx = bp_index(pc);
bp_table[idx].valid = 1;

if (taken && bp_table[idx].counter < 3)
    bp_table[idx].counter++;
else if (!taken && bp_table[idx].counter > 0)
    bp_table[idx].counter--;
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
    int res = 0;
    if (op == 0x4) res = a + b;
    if (op == 0x5) res = a - b;
    if (op == 0x6) res = a * b;
    if (op == 0x7) res = (b == 0 ? 0 : a / b);

    cpu.Z = (res == 0);
    cpu.C = (res > 255);
    return res & 0xFF;
}

int flush_pipeline = 0;

void fetch_stage() {
    ifex.IR = IMEM[cpu.PC++];
    ifex.PC = cpu.PC;
    ifex.valid = 1;

    if (is_branch(ifex.IR)) {
        int pred = predict_branch(cpu.PC);
        ifex.predicted_taken = pred;
        unsigned char operand = ifex.IR & 0xF;
        cpu.PC = pred ? operand : (ifex.PC + 1);
    }
}

void execute_stage() {
    if (!ifex.valid) return;
    unsigned char ir = ifex.IR;
    unsigned char op = (ir >> 4) & 0xF;

```

```
unsigned char operand = ir & 0xF;
instructions++;

switch (op) {
case 0x3:
    cpu.ACC = operand;
    break;
case 0x4:
    cpu.ACC += cpu.R[operand];
    break;
case 0x5:
    cpu.ACC -= cpu.R[operand];
    break;
case 0x6:
    cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6);
    break;
case 0x7:
    cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7);
    break;
case 0x0:
    cpu.R[operand] = DMEM[operand];
    break;
case 0xE:
    DMEM[operand] = cpu.R[operand];
    break;
case 0xA:
    cpu.PC = operand;
    jump_taken = true;
    break;
}
}
```